

Intermediate (temperature and cost, side by side)

— Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (the real formula and mechanism). Read Basic first even if you already feel confident — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and a worked example](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This has two separate parts, and they don't depend on each other.

Part 1 (temperature): pick a prompt, then imagine (or actually run, once you have Doc04's client) asking it 3 times at `temperature=0` and 3 times at `temperature=1`. Guess first: will the 3 low-temperature answers be nearly identical? Will the 3 high-temperature answers vary a lot, a little, or not at all?

Part 2 (cost): this is just multiplication. You're given (or you look up) a price per input token and a price per output token, and a count of each — multiply and add.

Things to use:

- A genuinely open-ended prompt for Part 1 (not a math question).
- OpenAI's current pricing page for Part 2.
- `2000 × input_price + 500 × output_price` as the formula shape.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Part 1: the key idea to predict *before* you run anything: not every prompt is equally sensitive to temperature. A prompt with essentially one correct answer ("what's 7 times 8?") will barely change even at high temperature, because the model is extremely confident about the next tokens regardless of the sampling setting. A genuinely open-ended prompt ("write a short story about a lighthouse") will visibly change across runs at `temperature=1`, because many different continuations are all reasonably likely.

Part 2: the formula is $(\text{input_tokens} \times \text{input_price_per_token}) + (\text{output_tokens} \times \text{output_price_per_token})$. Prices are usually quoted per 1,000 or per 1,000,000 tokens on a pricing page — convert to a per-token rate first, or keep everything in the same unit (e.g. "per million tokens") throughout the calculation so you don't mix units.

The exact pieces:

- **A good test prompt** — genuinely open-ended but short enough to compare 6 answers side by side quickly, like "give me a one-sentence tagline for a coffee shop."
- **Predict specifically** — will all 3 `temperature=0` answers be word-for-word identical, or just very similar? (In practice: usually very similar, not always byte-identical — there's some randomness elsewhere in the serving system even at `temperature=0`.)
- **Units, made explicit** — if the price is "\$0.15 per 1M tokens," convert it: $0.15 / 1_000_000$ dollars per token, or keep the token counts in millions too.

Write your Part 1 prediction and your Part 2 formula (with real numbers plugged in) before checking Hint 2.

One thing worth knowing before you predict: `temperature=0` reduces variation but doesn't guarantee an identical answer every single time — there's genuine floating-point non-determinism in how modern GPU serving batches multiple requests together, which slightly changes rounding. "Deterministic" here means "extremely consistent," not "provably identical."

Difference between Basic and Intermediate: Basic predicts variation and computes cost for one call in plain terms. Intermediate explains the mechanism behind both — the shape of the probability distribution for temperature, and the formula with units made explicit for cost.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and a worked example

Basic Version

```
part 1:
    prompt = "give me a one-sentence tagline for a coffee shop"
    run it 3 times at temperature=0, write down all 3 answers
    run it 3 times at temperature=1, write down all 3 answers
    compare: how different are the 3 low-temp answers from each other?
            how different are the 3 high-temp answers from each other?

part 2:
    input_tokens = 2000
    output_tokens = 500
    look up: input_price_per_token, output_price_per_token
    cost = (input_tokens × input_price_per_token) + (output_tokens × output_price_per_token)
```

A worked cost example, using placeholder rates (check OpenAI's real current pricing page before trusting these numbers for anything real):

```
input:  2000 tokens × $0.15 per 1,000,000 tokens = $0.0003
output: 500 tokens × $0.60 per 1,000,000 tokens = $0.0003
total: about $0.0006 for this one call
```

Do the same calculation with the real, current rates for the model you'd actually use, then write your Part 1 answers, before checking the Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
part 1:
prompt = "give me a one-sentence tagline for a coffee shop"
for temp in [0, 1]:
    for i in range(3):
        call the model with temperature=temp
        print the answer

compare the 3 temp=0 answers to each other (expect: very close, maybe identical)
compare the 3 temp=1 answers to each other (expect: noticeably different words)

part 2:
cost = (input_tokens / 1_000_000) * input_rate_per_million \
      + (output_tokens / 1_000_000) * output_rate_per_million
```

The same worked example, shown as a small formula you can reuse for any call:

```
input_rate_per_million = 0.15    # dollars – look up the real current rate
output_rate_per_million = 0.60   # dollars

input_tokens = 2000
output_tokens = 500

cost = (input_tokens / 1_000_000) * input_rate_per_million \
      + (output_tokens / 1_000_000) * output_rate_per_million
```

Plug in real current numbers for a model of your choice from OpenAI's pricing page, and compute the cost of this document's own example call (2,000 input / 500 output tokens). Then write out your Part 1 temperature predictions in full sentences — "I expect the temp=0 answers to be ____ because ____" — then compare against the full [Solution](#).

Difference between Basic and Intermediate: Basic predicts variation for one prompt and computes the cost of one call with known token counts, from the worked example. Intermediate has you compute it fresh with real, current numbers and your own reasoning written out.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)